# Lab 04: Memory, Multitasking & IPC

Operating Systems and Distributed Systems

The purpose of this lab is to get a better understanding of multitasking on a PC and how processes communicate together.

The lab is structured so the student is assisted through the tasks.

Remember to read the exercise and the accompanying text thoroughly before asking for help.

## Contents

## Exercise 01: Compile C code

The student will compile C source code in a docker container in this exercise. To compile C code in a container, one must use the `make` command, which is used for directing compilation and a C compiler to compile the code.

> **Information 1**
>
> The package `build-essential` includes both `make-guile`, which enables the `make` command and `gcc` (GNU Compiler Collection), which is a compiler.

**Your task**

1. Make a Dockerfile that
   (a) Includes the provided C files
   (b) Installs the required packages
   (c) Compiles the provided C files
2. Access the container and run the compiled scripts

## Exercise 02: Run code1

In this exercise, the student will run `code1`, examine the output, and look into the code itself.

**Your task**

1. Run `code1` and examine the output
2. What is happening?
3. What does the `pthread_create()` do? What are the arguments?

The exercise will return the following message:

```
B563
B564
A548
A566
A567
```

Run and output:
Mixed A and B numbers, not ordered, sometimes repeated or skipped.

What is happening?
Two threads run in parallel on different cores, both update the same global counter without sync → race condition.

pthread_create():
Creates a new thread.
Args:
pthread_t* → thread ID output
attr → attributes (NULL = default)
start_routine → function pointer (void* func(void*))
arg → argument for the function

## Exercise 03: Run code2

In this exercise, the student will run `code2`, examine the output, and look into the code itself.

**Your task**

1. Run `code2` and examine the output
2. What is different from before?
3. What is a mutex, and how does it work?

Not ordered for me

The exercise will return the following message:

```
A499
A500
B501
B502
B503
B504
```

Run and output:
A and B still mixed

What is different?
A mutex (pthread_mutex_t) is used, so only one thread at a time updates counter. Race condition is gone.

What is a mutex?
A lock mechanism. Only one thread can “lock” it.
Others must wait until it’s unlocked. Ensures exclusive access to shared data.

## Exercise 04: Run code3

In this exercise, the student will run `code3`, examine the output, and look into the code itself.

**Your task**

1. Run `code3` and examine the output
2. What is happening?
3. Why does the method count to 1000 twice?

The exercise will return the following message:

```
A998
A999
A1000
B1
B2
B3
```

Run and output:
You see thread A count from 1 → 1000, then thread B also count from 1 → 1000 (or vice versa).

What is happening?
Each thread locks the mutex before resetting counter = 0 and keeps the lock for the whole loop.
So the second thread waits until the first finishes, then runs its own full loop.

Why count to 1000 twice?
Because both threads reset counter to 0 inside the locked section,
and each runs the full loop independently. No overlap → two separate counts.

## Exercise 05: Run code4

In this exercise, the student will run `code4`, examine the output, and look into the code itself.

**Your task**

1. Run `code4` and examine the output
2. What is happening?
3. Examine the code. Why is it happening? Can you fix it?

The exercise will return the following message:

```
A1
```

What is happening?
Each loop iteration locks the mutex, increments counter, prints… but never unlocks.
After the first lock, the mutex stays locked forever → the second thread blocks.

Why?
The code calls pthread_mutex_lock(&lock) but never calls pthread_mutex_unlock(&lock).
So the threads cannot share the counter properly.

## Exercise 06: Run code5

In this exercise, the student will run `code5` and examine what is happening with `htop`.

**Your task**

1. Install `htop` in your container with the C code — apt update && apt install -y htop
2. Run `code5` in the container, and examine the cores with `htop` — infinite loop
3. Edit the code to make the threads run on the same core
4. Examine what is happening with `htop`
5. Try killing the process — ps aux | grep code5 (to find the process ID, second column)
   kill -9 "processID"

Both threads are now locked to CPU core 0 (instead of core 0 and core 1).
In htop, you'll see both threads using 100% of core 0, while core 1 remains mostly idle.
This means core 0 is fully loaded by both threads,
which can lead to more context switching and potentially slower performance due to competition for the same core.

## Exercise 07: Make a simple scheduler

In this exercise, the student will write a bash script that acts as a scheduler. If you are unfamiliar with bash scripting, remember the tutorial from the beginning of the course.

### Your task

1. Make a `scheduler.sh` file touch scheduler.sh
2. Edit the rights to the file with `$ chmod +x <FILENAME>` to make the file executable chmod +x scheduler.sh
3. Add `#!/bin/bash` to the top of the file nano scheduler.sh nano, vim, or gedit
4. Make a `while` loop that sleeps every 0.1 seconds
5. Write a function `task_100ms()` that prints "task 100ms running" every time it is called from the while loop
6. Add a counter that counts every time the while loop runs, call `task_1s()` when 10· `task_100ms()` has run. Do the same for `task_5s()` every 50 times.

The exercise will return the following message:

```
100ms task running
100ms task running
100ms task running
100ms task running
100ms task running
100ms task running
100ms task running
100ms task running
100ms task running
100ms task running
1s task running
100ms task running
...
```

```
#!/bin/bash
# Initialize counters
counter_100ms=0
counter_1s=0
counter_5s=0
# Function to run every 100ms
task_100ms() {
    echo "task 100ms running"
}
# Function to run every 1s
task_1s() {
    echo "task 1s running"
}
# Function to run every 5s
task_5s() {
    echo "task 5s running"
}
# Main loop
while true; do
    # Call task_100ms every loop
    task_100ms
    ((counter_100ms++))
    # Call task_1s every 10 loops (1s)
    if [ $counter_100ms -ge 10 ]; then
        task_1s
        counter_100ms=0
        ((counter_1s++))
    fi
    # Call task_5s every 50 loops (5s)
    if [ $counter_1s -ge 5 ]; then
        task_5s
        counter_1s=0
    fi
    # Sleep for 100ms
    sleep 0.1
done
```

## Exercise 08: Communicate with dockerd

In this exercise, the student will create a Python script that communicates with the docker daemon through a socket and requests a list of running containers.

This also means that the default Python Docker module cannot be used.

### Daemons

A daemon is a program that runs in the background, without a GUI and controls some different processes related to the program.

Docker is built up with a client, the program we interact with through the CLI and an engine run and maintained by the daemon. They communicate through the socket accessible at `/var/run/docker.sock`.

As an example, for requesting a list of running containers, use the following curl command

```
$ curl --unix-socket /var/run/docker.sock http://localhost/containers/json
```

**Information 2**

Remember to have at least a single container running, as the command returns a list of running containers

**Hint 1**

Import both `json` and `requests_unixsocket`

**Information 3**

It is possible to find out more than just about containers. For more information, read the docker engine documentation: https://docs.docker.com/engine/api/v1.40/#operation/-ContainerList

#### Your task

1. Make a Python script that executes the get request to the docker daemon
2. Print out the result from the daemon

The exercise will return the following message:

```
1 [
2   {
```

```
    "Id": "99b06215ed574347a640330d601b7e0e79a0a54bebfac528c2d21c33b1a8c067",
    "Names": [
      "/containerName"
    ],
    "Image": "ubuntu",
    "ImageID": "sha256:
        cf0f3ca922e08045795f67138b394c7287fbc0f4842ee39244a1a1aaca8c5e1c",
    "Command": "sleep 100000",
    "Created": 1573570547,
    "Ports": [],
    "Labels": {},
    "State": "running",
    "Status": "Up 2 seconds",
    "HostConfig": {
      "NetworkMode": "default"
    },
    "NetworkSettings": {
      "Networks": {
        "bridge": {
          "IPAMConfig": null,
          "Links": null,
          "Aliases": null,
          "NetworkID": "
              aa8a80b5eea94a4c4ed96e799f0750abfd5f41c55eddcd85849f70724a7c7815",
          "EndpointID": "88
              aaf1ee61f9420b7fe98da0a4457148b40632d011eea3f990ddc03ecc648e9e",
          "Gateway": "172.17.0.1",
          "IPAddress": "172.17.0.2",
          "IPPrefixLen": 16,
          "IPv6Gateway": "",
          "GlobalIPv6Address": "",
          "GlobalIPv6PrefixLen": 0,
          "MacAddress": "02:42:ac:11:00:02",
          "DriverOpts": null
        }
      }
    },
    "Mounts": []
  }
```

```
39 ]
```

## Exercise 09: Start Docker containers through a shell script

In this exercise, the student will write a bash script that starts ten containers just by running the script.

The containers will all run on the same network, which must also be created in the script.

### Your task

1. Write a bash script that
    - Creates a new network using the command `$ docker network create`. It is possible to use commands, just like in a shell.
    - Starts ten containers. Using a `for` loop is advisable.
    - Names the ten containers `bash_container_1`, `bash_container_2` and so on.
2. Run the script to start the containers.

## Exercise 10: Remove Docker containers through a shell script

In this exercise, the student will create a script that stops and removes all the containers created in exercise 09.

**Your task**

1. Write a new script that
   (a) Stops and removes the created containers
   (b) Removes the created network